

Line Rasterization

問題定義

在數學上，一條線段是連續幾何物件。若給兩個端點

$$P_0 = (x_0, y_0), \quad P_1 = (x_1, y_1),$$

則它們決定了一條連續線段

$$L(t) = (1 - t)P_0 + tP_1, \quad 0 \leq t \leq 1.$$

也就是

$$x(t) = x_0 + t(x_1 - x_0), \quad y(t) = y_0 + t(y_1 - y_0).$$

但螢幕上的 pixel 並不是連續平面，而是整數格點的離散集合：

$$(x, y), \quad \text{where } x, y \in \mathbb{Z}.$$

因此真正的問題不是「直線怎麼定義」，而是：

已知一條連續線段，如何選出一串整數格點，讓它們在視覺上最接近原本的線段？

連續直線通常不會剛好穿過整數格點。例如從 (2, 2) 到 (9, 5) 的線段，其斜率為

$$m = \frac{5 - 2}{9 - 2} = \frac{3}{7}.$$

當 $x = 3$ 時，理想的連續直線高度其實是

$$y = 2 + \frac{3}{7},$$

這不是整數。因此在 rasterization 時，我們必須在附近的整數格點之間做離散決策。

形式化定義：什麼叫做最接近直線

對於每一個要畫出的 pixel center (x, y) ，考慮它到理想直線的距離。若把直線寫成一般式

$$F(x, y) = ax + by + c = 0,$$

則點 (x, y) 到此直線的有號距離為

$$d(x, y) = \frac{ax + by + c}{\sqrt{a^2 + b^2}}.$$

分母 $\sqrt{a^2 + b^2}$ 對同一條線是常數，因此比較哪個 pixel 較接近直線時，只要比較

$$|F(x, y)|$$

即可，不必真的做平方根運算。Bresenham 的目標，就是在每一步只比較局部候選點，並用可遞推更新的整數誤差量替代浮點距離。

從兩點式得到隱式直線方程

給定端點

$$P_0 = (x_0, y_0), \quad P_1 = (x_1, y_1),$$

令

$$dx = x_1 - x_0, \quad dy = y_1 - y_0.$$

通過 P_0 與 P_1 的直線可寫成隱式方程

$$F(x, y) = dy (x - x_0) - dx (y - y_0) = 0.$$

展開後為

$$F(x, y) = dy x - dx y + (dx y_0 - dy x_0).$$

這個表示非常重要，因為：

- $F(x, y) = 0$ 表示點在直線上。
- $F(x, y) > 0$ 表示點位於有向線段 $P_0 \rightarrow P_1$ 的右側。
- $F(x, y) < 0$ 表示點位於有向線段 $P_0 \rightarrow P_1$ 的左側。

這個左右關係可以直接從二維外積看出來：

$$F(x, y) = \det((x - x_0, y - y_0), (dx, dy)).$$

也就是說， $F(x, y)$ 是從線段方向向量 (dx, dy) 轉到點向量 $(x - x_0, y - y_0)$ 的有號面積。

一般化觀點：選一個主軸，再在副軸做必要修正

要讓同一套邏輯適用於所有方向，最自然的方法是把線段分成：

- 主軸 major axis：絕對位移較大的那一軸。
- 副軸 minor axis：絕對位移較小的那一軸。

定義

$$adx = |dx|, \quad ady = |dy|, \quad sx = \text{sign}(dx), \quad sy = \text{sign}(dy).$$

其中

$$\text{sign}(u) = \begin{cases} 1 & \text{if } u > 0 \\ 0 & \text{if } u = 0 \\ -1 & \text{if } u < 0 \end{cases}$$

然後分兩種情況：

- 若 $adx \geq ady$ ，表示線主要沿 x 方向前進。
- 若 $ady > adx$ ，表示線主要沿 y 方向前進。

因此整個問題被統一成：每一步先沿 major axis 走一格，再判斷是否也要沿 minor axis 走一格。這種局部步進同時保證了 rasterized line 的鄰接性；後面會看到它自然形成 8-connected path。

x-major 的完整推導

先假設

$$adx \geq ady.$$

也就是 major axis 是 x 。為了保持一般性，不假設 dx 或 dy 的正負；方向全部交給

$$sx = \text{sign}(dx), \quad sy = \text{sign}(dy).$$

目前 pixel 設為 (x, y) ，下一步的兩個候選點是

$$E = (x + sx, y), \quad NE = (x + sx, y + sy).$$

這裡 E 只是「只沿 major axis 前進」的代號，NE 則是「major 與 minor 一起前進」的代號；即使 $s_y = -1$ ，幾何上也只是名稱保留，推導本身不變。

候選點的分界中點

若兩個候選點等距，分界應發生在它們的中點

$$M = \left(x + s_x, y + \frac{s_y}{2} \right).$$

因此 decision rule 應該是：

- 若直線在此中點的幾何位置更接近上方候選，選 NE。
- 若直線在此中點的幾何位置更接近下方候選，選 E。
- 若剛好通過 M ，則出現 tie，必須指定規則。

在第一象限的直觀圖像中，這可以直接看成中點位於直線哪一側；但對一般 octant，不應直接把 $F(M)$ 的正負號翻成固定的 E / NE 規則，因為符號還會受到 $s_x s_y$ 的影響。一般情況必須由後面的不變量接管。

Midpoint decision variable

把 M 代入

$$F(x, y) = dy (x - x_0) - dx (y - y_0),$$

得到

$$D = F(M) = dy (x + s_x - x_0) - dx \left(y + \frac{s_y}{2} - y_0 \right).$$

這個 D 就是 midpoint decision variable。

若要看得更直接，可以比較兩候選點的絕對值：

$$F_E = dy (x + s_x - x_0) - dx (y - y_0),$$

$$F_{NE} = dy (x + s_x - x_0) - dx (y + s_y - y_0).$$

由於

$$F(M) = \frac{F_E + F_{NE}}{2},$$

所以 $F(M)$ 的數值正好是兩候選點隱式值的平均。當它落在兩者之間時，midpoint test 與選較接近直線的候選 pixel 是等價的；至於一般 octant 下究竟哪個符號對應到 NE，必須結合 $s_x s_y$ 一起判讀。

初始值為什麼是半個 major step

現在把目前 pixel 代回起點 (x_0, y_0) 。此時第一步的中點是

$$M_0 = \left(x_0 + s_x, y_0 + \frac{s_y}{2} \right).$$

因此

$$D_0 = F(M_0) = dy \cdot s_x - dx \cdot \frac{s_y}{2}.$$

因為

$$dx = sx \cdot adx, \quad dy = sy \cdot ady,$$

所以

$$D_0 = sy \cdot ady \cdot sx - (sx \cdot adx) \left(\frac{sy}{2} \right) = sx \cdot sy \left(ady - \frac{adx}{2} \right).$$

這說明真正的 midpoint 初始 decision 值，本質上就是

$$ady - \frac{adx}{2}$$

乘上一個固定符號 $sx \cdot sy$ 。

而實作時更常用的整數誤差量直接記為

$$err = \frac{adx}{2} - ady.$$

它與 D_0 只差一個固定負號與符號因子，因此攜帶完全相同的決策資訊。這就是為什麼初始化會出現半個 major step：因為第一個 decision 正是在比較起點之後那一步的兩個候選點，而它們的分界點位於副軸的半格位置。

遞推增量從哪裡來

假設目前 decision variable 是

$$D = F\left(x + sx, y + \frac{sy}{2}\right).$$

若選 E，下一個 pixel 變成 $(x + sx, y)$ ，所以下一步的中點是

$$M_{E'} = \left(x + 2 \cdot sx, y + \frac{sy}{2}\right).$$

因此

$$D_{E'} - D = F(M_{E'}) - F(M) = dy \cdot sx,$$

也就是

$$D_{E'} = D + dy \cdot sx = D + sx \cdot sy \cdot ady.$$

若選 NE，下一個 pixel 變成 $(x + sx, y + sy)$ ，所以下一步的中點是

$$M_{NE'} = \left(x + 2 \cdot sx, y + 3 \cdot \frac{sy}{2}\right).$$

因此

$$D_{NE'} - D = F(M_{NE'}) - F(M) = dy \cdot sx - dx \cdot sy,$$

也就是

$$D_{NE'} = D + dy \cdot sx - dx \cdot sy = D + sx \cdot sy \cdot (ady - adx).$$

所以 midpoint variable 的增量只有兩種固定常數，這正是 Bresenham 可以用遞推完成的根本原因。

從 midpoint variable 到整數 err

上面定義的 D 含有 $\frac{1}{2}$ ，因為中點在半格位置。實作上不把整個式子乘以 2，而是直接改寫成等價的整數誤差遞推。

在 x-major 情況，一個常見選擇是定義

$$\text{err} = \frac{\text{adx}}{2} - \text{accumulated minor error}.$$

初始化時

$$\text{err}_0 = \frac{\text{adx}}{2}.$$

每走一個 major step，理想直線在副軸方向多累積 ady 的需求，所以

$$\text{err} \leftarrow \text{err} - \text{ady}.$$

若 $\text{err} < 0$ ，表示已經越過 half-step 分界，因此必須在副軸補一步，並把一整個 major-length 加回：

$$y \leftarrow y + \text{sy}, \quad \text{err} \leftarrow \text{err} + \text{adx}.$$

err 的形式化不變量

在 x-major 情況下，令目前 pixel 為 (x, y) ，並定義下一步分界中點

$$M(x, y) = \left(x + \text{sx}, y + \frac{\text{sy}}{2} \right).$$

若 loop 已經完成了 k 次 major step，且其中有 r 次做了 minor correction，則目前座標必為

$$x = x_0 + k \text{sx}, \quad y = y_0 + r \text{sy}.$$

此時，程式變數 **err** 在 loop 開始時滿足

$$\text{err} = \frac{\text{adx}}{2} - k \text{ady} + r \text{adx}.$$

接著程式會先做

$$x \leftarrow x + \text{sx}, \quad \text{err} \leftarrow \text{err} - \text{ady}.$$

此時拿來判斷是否要跨副軸的測試量為

$$\text{err}_{\text{test}} = \frac{\text{adx}}{2} - (k + 1) \text{ady} + r \text{adx}.$$

另一方面，把 loop 開始時的 pixel (x, y) 代回下一步中點

$$M(x, y) = \left(x + \text{sx}, y + \frac{\text{sy}}{2} \right)$$

可得

$$F(M(x, y)) = \text{sx sy} \left((k + 1) \text{ady} - r \text{adx} - \frac{\text{adx}}{2} \right) = -\text{sx sy} \cdot \text{err}_{\text{test}}.$$

所以在 $\text{if err} < 0$ 這一行真正被判斷的量，與 midpoint value $F(M(x, y))$ 只差一個固定因子 $-\text{sx sy}$ 。因此

- $\text{err}_{\text{test}} < 0$ ，等價於 $F(M)$ 與 $-s_x s_y$ 同號。
- 這也等價於：下一步應選 NE，而不是 E。

這就是一般 octant 下真正通用的決策規則；它不是單看 $F(M)$ 正負號，而是看 $F(M) = -s_x s_y \cdot \text{err}_{\text{test}}$ 這條不變量。

為什麼整數除法 $\frac{\text{adx}}{2}$ 不會破壞正確性

若 adx 是偶數，則 $\frac{\text{adx}}{2}$ 沒有問題。若 adx 是奇數，則實數上的半格其實是

$$\frac{\text{adx}}{2} = n + \frac{1}{2},$$

但程式中的整數除法會得到

$$\text{floor}\left(\frac{\text{adx}}{2}\right) = n.$$

這看起來少了 0.5，但不影響最終選出的 pixel，理由如下：

- 之後所有更新量 ady 與 adx 都是整數。
- 因此整個過程中的 err 永遠是整數。
- 真正的實數判斷是是否小於 0.5 的門檻。
- 對整數變數而言， $\text{err} < 0.5$ 等價於 $\text{err} \leq 0$ 。

而常見程式寫成 $\text{err} < 0$ ，代表它把平手 $\text{err} = 0$ 固定歸到不跨副軸那一邊。這就是 tie-breaking 規則，而不是數學錯誤。

Tie-breaking 規則

本文件採用的規則是：

if $\text{err} < 0$: 做 minor correction; else : 不做 minor correction.

因此當 $\text{err} = 0$ 時，演算法選擇：

- x-major：保留目前的 y 。
- y-major：保留目前的 x 。

也就是等距時偏向不跨副軸。

General Bresenham Algorithm

```
GENERAL-BRESENHAM((x_0, y_0), (x_1, y_1)):  
1  dx ← x1 − x0  
2  dy ← y1 − y0  
3  adx ← |dx|  
4  ady ← |dy|  
5  sx ← sign(dx)  
6  sy ← sign(dy)  
7  x ← x0  
8  y ← y0  
9  if adx ≥ ady then  
10 |   err ←  $\frac{adx}{2}$   
11 |   for t ← 0 to adx do  
12 |       plot (x, y)  
13 |       x ← x + sx  
14 |       err ← err − ady  
15 |       if err < 0 then  
16 |           y ← y + sy  
17 |           err ← err + adx  
18 |   else  
19 |       err ←  $\frac{ady}{2}$   
20 |       for t ← 0 to ady do  
21 |           plot (x, y)  
22 |           y ← y + sy  
23 |           err ← err − adx  
24 |           if err < 0 then  
25 |               x ← x + sx  
26 |               err ← err + ady
```

這份演算法之所以是 general，是因為它把所有特殊情況都吸收到三個抽象量：

- adx, ady：控制 major 與 minor 軸。
- sx, sy：控制前進方向。
- err：控制何時跨越副軸。

因此象限、斜率正負、起點終點順序，都不再需要各自背一套公式。

y-major 分支完全對稱：只要交換 x 與 y 的角色，並以 $\frac{ady}{2}$ 初始化即可。

一個具體例子

考慮

$$P_0 = (2, 2), \quad P_1 = (9, 5).$$

則

$$dx = 7, \quad dy = 3, \quad adx = 7, \quad ady = 3, \quad sx = 1, \quad sy = 1.$$

因為 $adx \geq ady$ ，所以沿 x 做 major step。初始

$$err = \frac{7}{2} = 3$$

以整數除法處理。遞推結果如下：

step	plot	err - ady	need $y + = 1$?	new (x, y)
0	(2, 2)	0	no	(3, 2)
1	(3, 2)	-3	yes	(4, 3)
2	(4, 3)	1	no	(5, 3)
3	(5, 3)	-2	yes	(6, 4)
4	(6, 4)	2	no	(7, 4)
5	(7, 4)	-1	yes	(8, 5)
6	(8, 5)	3	no	(9, 5)
7	(9, 5)	end	end	end

因此得到的 rasterized line 為

$$(2, 2), (3, 2), (4, 3), (5, 3), (6, 4), (7, 4), (8, 5), (9, 5).$$

更精確地說，在每個固定的 major-axis 位置上，演算法選出的 minor-axis 整數值，都是讓 $|F|$ 最小的候選值之一；若出現兩者等距，就依本文件指定的 tie-breaking 規則選其中一個。

正確性命題：它到底保證了什麼

對 x-major 情況，可把命題正式寫成：

對每一個 major-axis 座標

$$x = x_0 + k \, sx, \quad k = 0, 1, \dots, adx,$$

Bresenham 選出的 y_k 是所有整數候選中，使得

$$|F(x, y)|$$

最小者之一；若有兩個候選值等距，則依 tie-breaking 規則選定其中一個。

y-major 情況完全對稱。這說明它不是在求整條線的全域最短路徑，而是在每一個 major step 上都做局部最優的離散選擇。

連通性：為什麼是 8-connected

每一步中，pixel 只可能做以下兩種移動：

- 只沿 major axis 走一格。
- major 與 minor 同時各走一格。

因此相鄰兩個 pixel 的座標差只可能是

$$(sx, 0), \quad (0, sy), \quad (sx, sy).$$

這代表 Bresenham 產生的是 8-connected path，而不是 4-connected path。

在 line rasterization 中，這正是合理選擇，因為：

- 真實直線本來就可能穿過對角鄰接的 pixel。
- 若強制限制成 4-connected，只允許水平或垂直相鄰，線會出現多餘折角。
- 8-connected 能更自然地貼近歐幾里得直線的方向。

演算法性質

若線段長度的 major-axis 投影為

$$N = \max(|dx|, |dy|),$$

則 Bresenham 只需做 $O(N)$ 次迭代，並且除了少數整數變數外不需要額外記憶體，因此空間複雜度為 $O(1)$ 。

loop body 的核心操作只有：

- 加法。
- 減法。
- 比較。
- 符號判斷。

初始化階段仍需要：

- $|dx|, |dy|$ 的絕對值。
- $\text{sign}(dx), \text{sign}(dy)$ 的符號計算。
- 一次 $\frac{adx}{2}$ 或 $\frac{ady}{2}$ 。

但在主迴圈內不需要乘法、除法或浮點運算。

與 DDA 的比較

另一個常見方法是 DDA，其基本想法是

$$x_{k+1} = x_k + \frac{dx}{N}, \quad y_{k+1} = y_k + \frac{dy}{N}.$$

然後每一步再四捨五入到最近整數 pixel。

DDA 的優點是概念直觀；但缺點是它通常依賴浮點累積。當一條線需要走很多步，而每一步都做

$$y \leftarrow y + \frac{dy}{N},$$

則浮點表示誤差可能在長線段上逐步累積，最後某一步的 round 結果晚一格或早一格。相對地，Bresenham 的誤差量始終由整數遞推控制，並且每一步都只在兩個相鄰候選間做判斷；它的誤差不會以浮點近似的形式漂移。

邊界上通常希望起點與終點都被包含，因此迴圈次數是

$$\max(adx, ady) + 1,$$

其中這個 +1 來自於：

- adx 或 ady 表示的是相鄰 major step 的間隔數。
- 但我們要畫的是包含兩端點在內的格點數。

例如從 $x = 2$ 到 $x = 9$ ，雖然只有 7 個間隔，卻有 8 個整數位置：

2, 3, 4, 5, 6, 7, 8, 9.

另外，水平線與垂直線其實也是一般情況的自然特例：

- 垂直線： $adx = 0$ 。
- 水平線： $ady = 0$ 。

因此演算法不需要額外特判也能工作。當 $dx = 0$ 或 $dy = 0$ 時，要注意

$$\text{sign}(0) = 0,$$

這代表該軸不移動，正符合水平或垂直線的幾何意義。